



Using Tend

Every feature, the quick-add syntax, setting up a semester, Canvas import and the local model assist.

VERSION **0.53.0** 21 SEPTEMBER 2026 TEND-IOTA-JADE.VERCEL.APP

Everything the app does, from nothing. The README next to this file covers running and deploying it; this one covers using it.

What it is

A personal task app. Three things shape the rest:

Offline first. The app reads and writes a database inside your browser, never the server directly. Every edit lands locally and instantly, and a background sync pushes it up and pulls your other devices' changes down. With no network nothing degrades and nothing is lost.

One person. No sharing, no assignment, no collaborators.

Installable. Add it to your home screen and it runs like a native app, with its own icon and no browser chrome.

Getting around

Press **g** then a letter.

GROUP	VIEWS
Plan	Today g t , Upcoming g u , Calendar g c
Work	Inbox g i , Courses g o , Projects g p , Someday g s
Looking back	Logbook g l , Review g r
More	Tags g h , Views g v , Board g b , Focus g f
	Settings g ,

The four under More sit behind a disclosure in the sidebar, remembered per device. On a phone the bottom bar carries Today, Upcoming, Courses, Inbox and More.

Cmd+K opens the command palette, which reaches every view plus every course, project, tag and saved view by name. **?** lists every shortcut. **/** searches tasks.

Typing a task

One line does the whole job. **n** opens the field from anywhere.

```
Read chapter 4 friday 9am !p1 #reading +CS6260 @thesis
```

That becomes a task titled "Read chapter 4", due Friday at 9am, top priority, tagged `reading` , attached to course CS 6260 and project thesis. Everything the parser consumes leaves the title.

TOKEN	MEANING
<code>today</code> <code>tod</code> <code>tomorrow</code> <code>tmr</code> <code>tmrw</code> <code>yesterday</code> <code>yest</code>	the obvious
<code>monday</code> ... <code>sunday</code> , or <code>next friday</code>	the next one of those
<code>in 3 days</code> <code>in 2 weeks</code> <code>in 1 month</code> <code>next week</code>	relative
<code>3 days ago</code> <code>2 weeks ago</code>	backwards, for logging something done
<code>2026-09-30</code> <code>Sep 14</code> <code>14 Sep</code>	explicit
<code>9am</code> <code>9:30pm</code> <code>14:00</code> <code>noon</code> <code>midnight</code>	a time
<code>!p1</code> <code>!p2</code> <code>!p3</code>	priority, 1 highest
<code>!</code> <code>!!</code> <code>!!!</code>	the same three levels, fewer keys
<code>#tag</code>	a tag, created if new
<code>@project</code>	a project, created if new
<code>+CS6260</code>	a course

A bare month and day rolls forward, so `Sep 14` typed in October means next year. That is right for quick capture and wrong for a syllabus, which is why syllabus import resolves dates against the term window instead.

Working a list

KEY	DOES
<code>j</code> <code>k</code>	move the cursor
<code>Enter</code>	open the task
<code>x</code>	tick it
<code>Shift+S</code>	selection mode, for acting on many at once
<code>Cmd+Z</code>	undo the last change, as one gesture rather than one field

Drag a row to reorder it. On a phone, hold for a moment and then drag. Each list remembers its own sort, per device: manual, due date, priority, created, title, or pressure.

A task with a start date stays hidden until that date, collected under a "Starts later" disclosure at the foot of the list. This is how a November paper stops shouting at you in September.

Three ways in

1. **Type it.** **n** anywhere.
2. **Mail it.** Forward anything to your capture address and it lands in the Inbox. The subject is read exactly as the quick-add field reads it, the body becomes the note, and attachments are listed by name. Only mail from your own account address is accepted.
3. **Let Canvas bring it.** See below.

Anything with no home lands in the Inbox.

Setting up a semester

1. **g o** for Courses, then **New course**. There is no separate button for a term: the Term dropdown carries "Start a new term...", and picking it creates one guessed from today's date, **Fall 2026** running August to December if you are adding a course in September. You are not asked for exact semester dates on the way to your first course.
2. Press **Edit term** in the header and correct those dates to your registrar's. Do this before pasting any syllabus, because syllabus import resolves a bare **Sep 14** against this window.
3. Add the rest of your courses, picking the term you just made. Each one takes a code, title, credits, instructor, colour, and when it meets. Set the code to what you would want to type after **+**.
4. In each course's **Grades** tab, enter the weight components from the syllabus, for example Homework 30, Midterm 25, Final 30, Participation 15. Set "drop lowest" where the syllabus allows it.
5. **Settings, Coursework**: paste your Canvas feed URL and press Add, then Read the feeds.
6. **Settings, Your week**: set your daily capacity in minutes and which days you work, so workload numbers mean something.

A course with 0 credits is kept out of GPA while still collecting its deadlines, which is what you want for a non-credit shell like an orientation or a community space.

Grades

The Grades tab answers three questions.

Where you stand. A weighted mean over the components that have at least one graded item. A component nobody has marked yet is left out rather than counted as a zero.

Where you land. The same, with unmarked work projected at that component's current rate.

What you need. Set a target and it solves for the percentage required on everything still unearned, and says plainly when a target has stopped being reachable.

Weight splits by points, not by item count. Homework worth 30% over 500 points with 200 marked is 12% settled and 18% still moving. Term GPA weights each course's projection by its credits and tells you how many courses it managed to count, because a 4.0 over one of five courses is not a 4.0.

Canvas

Canvas publishes a private calendar feed. There is no API token and no OAuth.

1. In Canvas open **Calendar**, then **Calendar Feed** at the bottom right. Copy the `.ics` URL.
2. In Tend, **Settings, Coursework**, paste it and press Add.
3. Press **Read the feeds**.

After that it maintains itself. The daily cron re-reads every enabled feed, so a new deadline appears whether or not you opened the app.

Assignments become tasks. Lectures, exams and office hours become read-only events drawn behind the day on Today and the Calendar, so your free hours are visible.

Each item is matched to a course in three passes: an explicit feed label, then your course code appearing inside what Canvas wrote, then the reverse for a feed that abbreviates. Canvas writes `CS-6260-A` where you would type `CS 6260`, and folding to letters and digits handles that. Two courses matching the same item resolves to nothing rather than a guess, and the item waits in the Inbox where you can see it.

Three rules hold across re-reads, enforced in SQL rather than in the route:

- Importing twice does not duplicate. Identity is the Canvas UID.
- Your edits win. Each imported task stores a snapshot, and the feed may overwrite a field only while that field still holds what the feed last wrote.
- A deleted task stays deleted. The feed cannot resurrect it.

The feed URL is credential-shaped: anybody holding it can read the whole calendar. It is masked to its host once saved, never logged, and never returned by the import route. Treat it like a password, and reset it in Canvas if it leaks.

What the feed cannot give you is anything an instructor left out of Canvas. That is the next section.

Syllabus import

Open a course and paste the grading schedule straight out of the syllabus. One item per line, or a table copied out of a PDF. Every line runs through the same parser as quick-add, then you get an editable preview with title,

date, component and points. Confirm and the whole import commits as one batch, so one bad row does not leave you half-imported.

Dates resolve against the term window rather than rolling forward, so `Sep 14` in a Fall 2026 syllabus means September 2026 even when you paste it in November.

The local model assist

If you have Ollama running, a button appears that fills the grid in for you. It reads prose a plain parser cannot, for example "the second project is due the Friday before fall break".

Three things are true of it by design:

- **The model never writes a task.** It fills the same form you would have filled, and you confirm it. A machine-extracted deadline gets a human check.
- **Nothing runs when you are not using it.** Tend starts no server and holds no connection. It sends one request per press, with `keep_alive: 0` so the weights unload the moment the answer comes back. Nothing sits in RAM between imports.
- **Nothing leaves your machine.** The request goes to loopback. No API key exists anywhere in the app.

The default model is `qwen3.5:4b`, which is enough: a syllabus table is extraction, not reasoning, and a JSON schema does the structural work. The model name is a settings field, so a bad result can be answered by pointing at a bigger model.

Starting Ollama. Nothing to leave open and nothing to configure, as long as the Ollama server is up:

```
ollama serve          # or just open the Ollama app once
ollama pull qwen3.5:4b
```

You do not need to load or warm the model. Tend names it per request and Ollama loads it on demand.

It only works when Tend is served from localhost. A request from an HTTPS page to a loopback address is a private-network request, which browsers gate behind a preflight Ollama does not answer, and the call hangs rather than failing. Measured against the deployed app: still pending after two minutes. So the probe gives up after 1.5 seconds and the button simply never appears on the deployed site. Run `npm run dev` and use `http://localhost:3000` when you want the assist.

A failed probe is the ordinary case, not an error. The row reads "not reachable" and the button is absent. No toast, no retry, no spinner waiting on a daemon nobody started. On a phone there is no Ollama at all and the manual grid is the whole feature, which is the degradation working.

Workload

Put a time estimate on a task and it starts counting. Given your daily capacity and working days, Tend reports each day as light, full or over.

The number that earns its place is slack: the working hours available before a deadline, minus the estimates of everything due on or before it. Negative slack means the deadline is already gone, and it says so three days early instead of on the morning. Slack belongs to a day rather than a task, so two things due the same day share one figure, because they compete for the same hours.

A task with no estimate contributes nothing and is counted out loud, so the strip says "2 with no estimate" rather than inventing a duration.

Surfaces: a capacity strip across Upcoming, a load pip per Calendar cell, today's load in the Today header, an overdrawn badge on a row, a Pressure sort, and an Unplanned tray beside the Calendar that makes planning a week the same gesture as dragging.

Reminders

Email runs off a SQL pipeline: a morning digest of what is due, a nudge for what slipped, and a weekly review. Times are wall clock in the timezone set in Settings, so changing timezone does not silently reschedule your mornings. Every email carries a link that turns that kind off without signing in.

Per-task reminders sit in the detail panel, which is how "remind me two days before" gets said. The global lead time cannot express that.

Web push is available too, per device.

Your day, as a student

Today carries a strip of the nearest deadline per active course with a day count, plus today's classes read off each course's meeting times. Courses carries a three-week exam radar. Review carries focused minutes per course and points landed against points available, with time that cannot be attributed to a course said out loud rather than spread around.

Sync and devices

Sign in with an emailed code. Every device holds its own full copy and converges per field, so completing a task on your phone while retitling it on your laptop gives you a completed and retitled task. Signing out keeps anything not yet pushed.

Some settings are per device on purpose and never sync: theme, list sorts, the local model endpoint, and push subscriptions. A model endpoint is a property of a machine.